

Tutorial Lab 2: Prelab Summary

Tutorial Lab 2: Error Propagation—this tutorial lab introduces you to how to assign error to quantities that are calculated from measured quantities.

The Necessity of Error Propagation

As you saw in Tutorial Lab 1, there is uncertainty present in all measurements. This uncertainty is referred to as the *error* in the measurement. To reflect this, measured values are reported as (value $\pm$ error). Often, you need to do further calculations using the measured values to determine a final parameter of interest. When measured values are combined (e.g. added, multiplied, etc.), their errors must also be combined to give an error on the final calculated value. *Error propagation* is the process of determining the error in a final result from the errors of the measured values.

Below is a basic summary of what you need to know for the lab. For further details, consult the document *Measurement Error and Data Analysis* on onQ under Reference Materials.

Simplest Case: Result Depends on a Single Variable

Let's first consider the case where we only need to measure a single variable x to determine our desired result y . If the measurement of x has an error δx , then the error in y will depend on the error in x and how quickly y changes with x , i.e. the slope or derivative of the y vs. x graph, as seen in Figure 1.

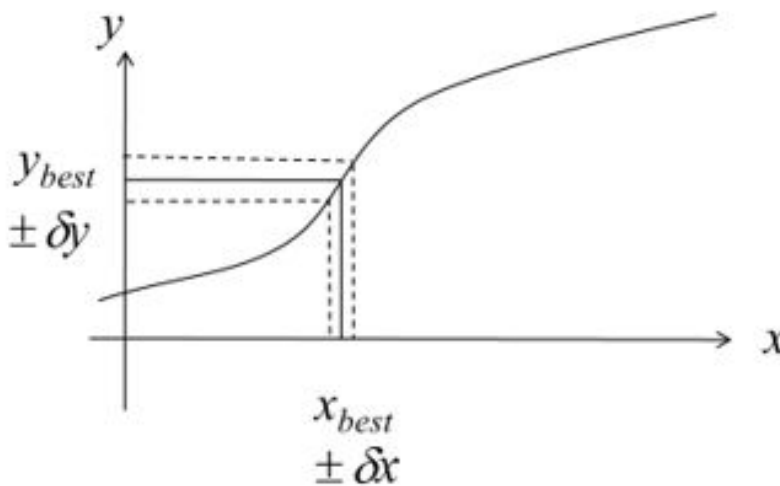


Figure 1: Example graphical result showing the relationship between δx and δy .

As long as the uncertainty is small enough that the graph can be treated as approximately linear in the range around x_{best} , then the uncertainty in y is given by

$$\delta y = \left| \frac{dy}{dx} \right| \delta x. \quad (1)$$

Maximum Uncertainty for a Function of Two or More Variables

We can extend Equation 1 to cases where a value is calculated based on measurements of several variables, say $x_1, x_2, x_3, \dots$. In this case, the *maximum error* on the calculated quantity y can be estimated by adding up the error contributions of each of the measured quantities:

$$\delta y = \left| \frac{\partial y}{\partial x_1} \right| \delta x_1 + \left| \frac{\partial y}{\partial x_2} \right| \delta x_2 + \left| \frac{\partial y}{\partial x_3} \right| \delta x_3 + \dots \quad (2)$$

Here, $\partial y / \partial x_1$ is the partial derivative of y with respect to x_1 and is found simply by differentiating y with respect to x_1 while treating all other variables as constants.

Consider the hypothetical case where we need to calculate the speed of an object based on the time it took to travel a known distance. If the time is measured to be $t = 18.7 \pm 0.2$ s, and the distance is measured to be $D = 64.0 \pm 0.5$ m, then the best estimate for the speed is $v = D/t = 64.0/18.7 = 3.42$ m/s. The error on the speed $v = D/t$ is calculated as follows:

$$\begin{aligned} \delta v &= \left| \frac{\partial v}{\partial D} \right| \delta D + \left| \frac{\partial v}{\partial t} \right| \delta t \\ \delta v &= \left| \frac{1}{t} \right| \delta D + \left| -\frac{D}{t^2} \right| \delta t \\ \delta v &= \left| \frac{1}{18.7} \right| (0.5) + \left| -\frac{64.0}{18.7^2} \right| (0.2) = 0.063 \text{ m/s}, \end{aligned}$$

so we would report the speed as $v = 3.42 \pm 0.06$ m/s.

Better Estimate of Uncertainty by Adding Error in Quadrature (Pythagoras)

We usually assume that the measurement errors $\delta x_1, \delta x_2, \dots$ are random and independent and so they can be expected to cancel each other to some extent. This means that Equation 2 will usually *overestimate* the uncertainty δy . A more realistic (and smaller) estimate of the uncertainty is found by replacing the sum in Equation 2 with the one shown in Equation 3. This is called taking the sum “in quadrature.” It is basically like adding perpendicular vectors using the Pythagorean theorem.

$$\delta y = \sqrt{\left(\frac{\partial y}{\partial x_1} \delta x_1 \right)^2 + \left(\frac{\partial y}{\partial x_2} \delta x_2 \right)^2 + \left(\frac{\partial y}{\partial x_3} \delta x_3 \right)^2 + \dots} \quad (3)$$

Equation 3 gives an expression of the “likely” error while Equation 2 is more like a maximum error. Equation 3 is used in the Python program you will be downloading and using for future lab work in this course.

Sensitivity Analysis

At the end of an experiment, you should think about how you could improve different parts of the experiment to reduce the error. To know what to focus on for improving your

measurement, you can conduct a *sensitivity analysis*. This looks at each of the variables in your experiment and determines how much each contributes to the total error. Those with the largest contributions to the error are the ones you should focus on to improve the experiment.

The sensitivity calculation is related to Equation 3. To determine the fractional contribution of variable x_1 to the total error, we simply take the first term in that equation and divide it by the total error squared, and then multiply by 100% to give us a percentage. That is,

$$\% \text{ contribution from } x_1 = \left(\frac{\frac{\partial y}{\partial x_1} \delta x_1}{\delta y} \right)^2 \times 100\% \quad (4)$$

Equation 4 is determined automatically by the Python code that you will be using to conduct a sensitivity analysis and produce a graph of the relative contributions for each variable. You should be able to interpret the results of the code.

Downloading the Python Code

Error propagation involving multiple variables can be tedious. For these calculations, you may use the provided Python code. When you run the code, it will ask you for your formula and variable values, with errors. Then, it will calculate the propagated errors and produce the sensitivity analysis for you in the form of a bar chart.

Note that the Python code may be used for a function of multiple variables, or for a function of just one variable. If there is only one variable with measurement error, then the sensitivity analysis is not useful and should not be reported, since 100% of the error will come from that one variable.

Download and install Python via Anaconda. Download the error code. On onQ, watch the *Python Error Propagation Code Tutorial* video. Read the *Python Error Propagation Code Tutorial* file, which describes how to use the code for error propagation.